Decentralised Lending Service — Project Report

This project implements a lightweight decentralised lending service on Ethereum using a common funding pool, per-loan management contracts, and a trusted Bitcoin liquidity oracle. The system follows the course specification: contributors deposit ETH, a borrower creates a loan proposal, contributors vote by disposable value, and the loan is executed only after the proposal period and liquidity checks are satisfied.

Main implementation choices

- FundingPool keeps a total contribution map and an unlocked-balance view to enforce contribution lock semantics and support weighted voting by disposable value.
- Each approved proposal deploys a dedicated LoanContract so state and repayment logic remain isolated per loan.
- BitcoinOracle stores the latest BTC balance for durable oracle data and charges a minimum fee for updates to prevent spam and abuse.
- The project includes Hardhat tests and small Python wrappers to satisfy the PDF requirement for local setup, demonstration, and validation.

Gas measurement summary

- contribute: 67543 gas
- createProposal: 184611 gas
- voteYes: 78346 gas
- executeLoan: 716057 gas
- repayLoan: 36890 gas

Reentrancy demonstration

The project adds a deliberately vulnerable contract (ReentrancyVulnerableLoan) and a malicious contract (ReentrancyAttacker). The exploit is a classic reentrancy issue: the victim sends ETH before clearing the account balance, allowing the attacker to re-enter withdraw() before the state update executes. The attached Hardhat test demonstrates that the attacker can drain more than the original deposit.

Malicious strategy discussion

A contributor can gain an unfair remuneration without reentrancy by concentrating disposable value and voting power. Because weighted approval is proportional to disposable value, a large actor can dominate proposal outcomes and later choose to hold or release votes strategically. With a colluding borrower or repeated small loans, the same actor can also amplify the compensation pool claims: if the actor controls several addresses or a large locked portion, they may withdraw a disproportionate share of the compensation pool after defaulted loans. This is not a reentrancy bug, but a governance concentration issue in a voting system grounded on disposable-value weighting.

User manual

- Install Node 22+ and run npm ci in the project root.
- Run npm run compile to compile the Solidity contracts.
- Run npx hardhat test to execute the included tests and gas checks.

- Run python3 scripts/setup_local_service.py and python3 scripts/demo_local_service.py for the local setup and example workflow.
- Use scripts/gas_report.js to print the gas summary for key operations.